How to Pick the Model: A Framework for Budget-Aware Orchestration

Anonymous Author(s)
Affiliation
Address
email

Abstract

Agent systems on verified tasks invite a deceptively simple question: which backbone model should be run? The strongest model is not always the best deployment choice, because verified progress must be bought under finite wall-clock, token, API, and evaluation budgets. We formulate this problem as deployment-aware model selection. A fixed router observes a pre-deployment information state and chooses a worker or harness action; the decision is scored by a checker-indexed deployment loss rather than by routing accuracy or final checker score alone. The key estimand is paired: on the same task instances, with the same router and action menu, does richer information change the selected action and reduce net deployment loss after its own measurement and context costs are charged? We instantiate the framework on an Karpathy's AutoResearch-inspired CIFAR-10 edit–verify benchmark, where a prompt-only router chooses among heterogeneous workers before costly deployment runs. The protocol reports when checker-score rankings disagree with deployment-loss rankings, when pre-deployment information changes routing, and whether those changes pay for themselves. The resulting protocol evaluates not which agent is best in isolation, but when information is worth buying before deploying one.

1 Introduction

A team building an agent system faces a practical question: for a concrete externally checked task instance, a finite menu of deployable actions, and deployment budgets, which action should be launched? The common answer is benchmark racing. Run a lower-cost worker and a higher-capability worker on held-out instances; choose the one with the best average score. For externally checked agentic tasks, this can be the wrong unit of comparison. A model that eventually succeeds after a long expensive search may be dominated by a lower-cost action that reaches a sufficient checked outcome earlier, or by a router that sends different instances to different workers.

This paper studies model and harness choice through *performance accounting*. The motivating regime includes code editing against unit tests, repository repair against continuous-integration checks, theorem search against a formal checker, and AutoResearch-style loops in which an agent edits a training program and receives external validation feedback [Jimenez et al., 2024, Karpathy, 2026]. These tasks are transductive in Vapnik's sense [Vapnik, 1996]: the system is handed a concrete instance and must produce a concrete artifact. They are also externally checked: progress is determined by a verifier or scoring procedure outside the model. In this regime, the deployment object is not standalone accuracy. It is the resource needed to reach, evaluate, and maintain checked progress, matching the time-centric view of agents as task solvers in Achille and Soatto [2026].

The central difficulty is that one benchmark score conflates mechanisms that imply different engineering decisions. A system can win because its worker has lower per-step cost. It can win

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

because the selected worker is more competent on a solver-relevant task mode. It can win because a pre-deployment signal reveals which mode the instance belongs to. Finally, a router may either exploit that information or waste it through mismatched allocation. The theory in this paper isolates these four mechanisms in an exact log-effort identity. The experimental protocol then asks whether the same mechanisms survive contact with measurable deployment losses, checker costs, router costs, and repeated stochastic runs.

The draft is organized around four contributions.

1. **Four-term performance accounting.** Under a packed latent-mode abstraction, Theorem 1 decomposes improvement in expected certified log-effort into cost, competence, routing information, and routing mismatch.
2. **Operational diagnostics.** We show how the four theorem terms are estimated from checked trajectories: competence from first threshold crossing, cost from the same routed trajectory, information from the router-visible signal, and mismatch between the mode structure exposed by that signal and the worker allocation induced by the router.
3. **Paired router evaluation.** For a fixed router and action menu, richer information is valuable only when it changes selected actions in a way that lowers paired deployment loss after measurement and context costs are charged.
4. **Executable-artifact benchmark protocol.** We instantiate the framework on a CIFAR-10 AutoResearch-style edit–verify benchmark with finite diagnostic states, a prompt-only router, a finite worker menu, admissible pre-deployment measurements, negative controls, and residual diagnostics for the theorem.

An externally checked task specifies the problem class and checker. Task modes are the finite types of that task: subpopulations that differ in solver-relevant structure, cost profile, or likely intervention. Task instances are concrete realizations within those modes, such as a particular starting artifact, data/regime specification, seed, budget, and checker configuration. In the operational benchmark, modes can be implemented as task-family, dataset-regime, architecture, seed-bin, baseline-probe bin, or crossed diagnostic states. Workers such as `gpt_5_3_codex_spark`, `gpt_5_3_codex`, `gpt_5_4`, or fixed agent policies are deployable actions that attempt those instances. This separation makes the four-factor identity testable: $p(a, s)$ measures how an action performs on a type of task instance, rather than reusing the action label as the state.

2 Verifiable agent design

We first state the theory-facing objects. The notation follows the original performance-accounting setup, but we use the deployment language needed for the operational protocol.

Definition 1 (Budgeted externally checked task)**.** *A budgeted externally checked task is a tuple*

$$\mathcal{T} = (\mathcal{X}, \mathcal{Y}, V, B_{\mathrm{dep}}, \mu).$$

Here $\mathcal{X}$ is an instance space, $\mathcal{Y}$ is an artifact or solution space, $V : \mathcal{X} \times \mathcal{Y} \to \{0, 1\}$ is an external verifier, B_{dep} is a deployment budget, and μ is the deployment distribution over instances. An agent succeeds on $x \sim \mu$ if it produces y with $V(x, y) = 1$ before exhausting B_{dep}.

The verifier may be binary, as in tests or proof checking, or scalar, as in validation loss or reward after thresholding. The key requirement is that success is evaluated outside the model. The task is transductive because performance is measured on concrete pairs (x, y) rather than on a learned population predictor.

Definition 2 (Workers, actions, and routed designs)**.** *Let $\mathcal{M} = \{M_1, \ldots, M_K\}$ be a finite menu of workers. A worker is any deployable solver that can operationally attempt the task instance and return an artifact for verification: a single model, or a fixed model-based agent policy that may call tools or other models according to a predeclared protocol. Let $\mathcal{A}$ be a finite menu of deployable actions. In the simplest worker-selection setting, $\mathcal{A} = \mathcal{M}$; more generally, an action includes the worker together with fixed stopping, retry, tool-use, submodel-call, and carryover rules. A routed design is a tuple*

$$d = (\mathcal{A}, h, R, \kappa),$$

where h is the harness protocol, R is a start-of-run router that selects an action after observing permitted information, and $\kappa(a) > 0$ is the per-unit deployment cost convention for action $a \in \mathcal{A}$. When no confusion is possible, we write M for the deployed worker or fixed action selected by the design. In the main V3 experiment, the orchestrator R and menu $\mathcal{A}$ are fixed. A routing policy is therefore the signal-conditioned map induced by a progressive signal protocol,

$$\rho_j : x \mapsto \hat{a}_R^j(x) = R(Z_j(x)) \in \mathcal{A}.$$

Future variants may also compare different orchestrators R, but the present protocol isolates the value of changing what the same orchestrator is allowed to observe before choosing a worker.

The router is part of the deployed system, but it is evaluated through the action it selects. In the main protocol, the router observes information before deployment, chooses one worker, and then the chosen worker performs the full edit–verify trajectory from the original task state. The selected worker does not see measurement records unless that carryover is explicitly declared as part of the deployable action.

Definition 3 (Task instances and task modes)**.** *Given $\mathcal{T} = (\mathcal{X}, \mathcal{Y}, V, B_{\mathrm{dep}}, \mu)$, a task instance is a concrete realization $X = x \in \mathcal{X}$ of that same task. It is the measurable object supplied to the deployed system, such as a starting artifact, data/regime specification, seed, budget, and checker configuration; the model and router may access only the fields made available by the protocol. A task mode is a latent variable*

$$S = \sigma(X) \in \mathcal{S} = \{1, \ldots, m_S\}$$

induced by a measurable map $\sigma : \mathcal{X} \to \mathcal{S}$ that assigns each concrete instance to one of finitely many solver-relevant variants of the task. It defines a prior $\pi_s = \mathbb{P}_\mu[S = s]$ and mode-conditional instance distributions $\mu_s = \mu(\cdot \mid S = s)$. Modes are therefore not different tasks and not worker labels. They are variants of the same task: finite subpopulations of instances that share solver-relevant structure and may favor different workers or interventions.

The mode variable is retained by the evaluator for accounting and diagnostic comparisons. It need not be observed by the worker, and in the main router experiment the router only sees permitted pre-deployment information. This lets the same theory cover both expert-defined task types and measurable diagnostic states used in the current benchmark.

Assumption 1 (Packed latent-mode family)**.** *Each instance $X \sim \mu$ is associated with a mode $S = \sigma(X)$ in a finite mode set $\mathcal{S}$, with prior π. In the main protocol, the unit of deployment is a full routed trajectory: the router chooses one worker before deployment, and that worker runs the edit–verify process up to horizon H. Conditional on mode s, worker or action M has certified first-success probability $p_M(s) > 0$ under the declared checker, budget, horizon, and success threshold. This probability is trajectory-level, but it is estimated from the step-wise checker trace by asking whether the threshold is reached at least once before the horizon. At the start of a run, the router observes a signal $Z = z$ and selects one action for the whole trajectory. The evaluator defines the mode distribution induced by that signal,*

$$\pi_z(s) = \mathbb{P}[S = s \mid Z = z].$$

The router is not required to output π_z or any diagnostic-state distribution. Instead, after the selected-worker runs are scored, the evaluator constructs a mode-facing allocation summary $q_z \in \Delta(\mathcal{S})$, with support containing the support of π_z. This summary records whether the worker chosen under signal z is aligned with the modes on which that worker has a favorable cost–competence profile. Write M_z for the worker or fixed action selected by the routing policy after observing z. For an instance whose true mode is s, the certified effort surrogate is proportional to

$$T_{\rho,q}(s, z) \propto \frac{\kappa(M_z)}{p_{M_z}(s)\, q_z(s)}. \tag{1}$$

Assumption 1 is deliberately restrictive. Real task instances can vary in infinitely many ways, but an empirical deployment campaign can only estimate and reuse finitely many state-conditioned quantities. The assumption says that, for the purposes of model selection, those concrete instances can be packed into a finite set of modes with stable worker success probabilities. The purpose is to isolate the regime in which cost, competence, information, and mismatch are algebraically separable and empirically testable. The operational campaign therefore reports residuals, hazard curves, and occupancy diagnostics to measure when this finite packing is too coarse.

Definition 4 (Cost-adjusted certified log-effort objective)**.** *For a fixed routed policy $\rho : Z \mapsto M_Z$ satisfying Assumption 1, define*

$$\mathcal{J}(\rho, q) = \mathbb{E}_Z[\log \kappa(M_Z)] + \mathbb{E}_Z \mathbb{E}_{S\sim\pi_Z}[-\log p_{M_Z}(S)] + \mathbb{E}_Z[\mathrm{H}(\pi_Z) + \mathrm{KL}(\pi_Z \| q_Z)]\,. \quad (2)$$

The design-selection problem is

$$d^\star \in \arg\min_{d\in\mathcal{D}} \mathcal{J}(\rho_d, q_d), \quad (3)$$

where each design d induces a routed policy ρ_d and allocation summary q_d, and lower $\mathcal{J}$ means lower expected certified log-effort. When the same orchestrator is evaluated under several progressive signals, $\mathcal{J}$ is applied to the signal-conditioned policy summaries; Appendix C writes this comparison explicitly for Z_j versus Z_0.

The worker/action quantities in (2) are $\kappa(M_Z)$ and $p_{M_Z}(s)$: how costly the selected worker is and how often it succeeds on mode s. The router and information quantities are Z, π_z, and q_z: what the router is allowed to observe, what that signal reveals about the task mode, and whether the worker chosen by the router is empirically appropriate for those modes.

Remark 1 (Meaning of the effort surrogate)**.** *Equation* (1) *says that, on a true mode s after signal z, certified effort increases with worker cost, decreases with worker success probability on that mode, and increases when the induced allocation summary gives too little diagnostic mass to that mode. The router is not literally routing jobs to modes; modes are evaluator-defined task types. The quantity $q_z(s)$ records whether the router's worker decision is aligned with the mode that actually governs the instance, and the KL term below charges the mismatch between signal-exposed mode structure and the induced worker allocation.*

3 Performance accounting

For a fixed signal value z, the routing part of the objective is the cross-entropy between the signal-induced mode distribution and the induced worker-allocation summary:

$$\mathrm{CE}(\pi_z, q_z) = -\sum_{s\in\mathcal{S}} \pi_z(s) \log q_z(s) = \mathrm{H}(\pi_z) + \mathrm{KL}(\pi_z \| q_z). \quad (4)$$

The KL term is a divergence over finite task-mode distributions. It is not a token-level language-model divergence.

Theorem 1 (Four-term routed-policy decomposition)**.** *Under Assumption 1, compare a prior-matched baseline policy ρ_0 that deploys M_0 without using Z and has allocation summary $q_z = \pi$, with a deployed routed policy summarized by (ρ, q). The improvement in expected certified log-effort, $\Delta = \mathcal{J}(\rho_0, \pi) - \mathcal{J}(\rho, q)$, decomposes as*

$$\Delta = \underbrace{\mathbb{E}_Z\left[\log \frac{\kappa(M_0)}{\kappa(M_Z)}\right]}_{\textit{cost}} + \underbrace{\mathbb{E}_Z \mathbb{E}_{S\sim\pi_Z}\left[\log \frac{p_{M_Z}(S)}{p_{M_0}(S)}\right]}_{\textit{competence}} + \underbrace{\mathrm{I}_\pi(S; Z)}_{\textit{routing information}} - \underbrace{\mathbb{E}_Z[\mathrm{KL}(\pi_Z \| q_Z)]}_{\textit{routing mismatch}}. \quad (5)$$

Proof sketch. Taking logs in (1) gives $\log T_{\rho,q}(s, z) = C + \log \kappa(M_z) - \log p_{M_z}(s) - \log q_z(s)$. Conditioning on $Z = z$ and averaging over $S \sim \pi_z$ converts $-\mathbb{E} \log q_z(S)$ into the cross-entropy in (4). Subtracting the prior-matched baseline cancels constants and uses $\mathrm{H}(\pi) - \mathbb{E}_Z \mathrm{H}(\pi_Z) = \mathrm{I}_\pi(S; Z)$. The full algebra is given in Appendix A. □

The four terms have different operational meanings. If the cost term is large and positive, a lower-cost worker may be preferable even when it reaches the first success later in step count. If the competence term varies by mode, a global worker ranking is likely to hide comparative advantage. If information gain is high, the pre-deployment state exposes solver-relevant structure. If mismatch is high, the router sees useful structure but fails to allocate computation according to it. In the V3 experiment, this is read as a comparison between signal-conditioned routing policies for the same orchestrator. Changing from Z_0 to Z_j is useful only if it changes the selected worker toward a better mode-conditional cost–success tradeoff, and that improvement survives the measured cost of obtaining the richer signal.

Figure placeholder. Final figure: left-to-right accounting pipeline. Inputs are checked task instances, finite diagnostic states, worker or harness actions, admissible pre-deployment information, and cost logs. The center computes the four log-effort terms. The right panel reports deployment-loss validation: rank flips, paired router gain, negative controls, and residuals.

Figure 1: The V3 framing keeps the four-term identity as the theory-facing diagnostic and tests it with deployment-loss outcomes.

Operational reading. The diagnostic state S is not an oracle label for the best worker. The evaluator fixes a candidate mode schema before the campaign, then tests whether that schema is useful by running workers and estimating the mode-conditional quantities $p(a, s)$ and $\kappa(a)$. Routing information asks whether a progressive signal makes those predeclared modes more identifiable. Routing shift asks whether the same orchestrator actually changes its worker choice when the signal is enriched. Routing mismatch is then evaluated retrospectively: after worker outcomes are logged on a diagnostic split, we ask whether the signal-induced choice was closer to the empirical mode-conditional cost–success frontier. The final decision is not made by the information term alone; it is made by paired deployment gain on held-out instances after measurement and context costs are charged. Appendix B gives the full operationalization of these diagnostics.

3.1 When lower-cost retries win

Theorem 2 (Single-mode retry crossover)**.** *On a single-mode task with a binary thresholded checker, fix the success threshold δ and horizon H. In this theorem, one attempt means one fresh deployment trajectory from the original task state, not one edit step inside a trajectory. Let a strong action have one-attempt first-passage success probability $p_h = \mathbb{P}[\tau_\delta(h, X) \le H]$ and expected worker-side first-passage cost $\kappa_h = \mathbb{E}[C_h(\tau_\delta(h, X) \wedge H)]$. Let a lower-cost action have success probability $p_l < p_h$ and cost $\kappa_l < \kappa_h$, with $0 < p_l < p_h < 1$. The number of lower-cost attempts needed to match the strong action's one-attempt first-passage success probability is*

$$D_{\mathrm{cross}} = \frac{\log(1 - p_h)}{\log(1 - p_l)}.$$

The corresponding integer fixed-depth retry block is cost-favorable whenever

$$\lceil D_{\mathrm{cross}} \rceil \, \kappa_l < \kappa_h.$$

If deployment stops after the first lower-cost successful trajectory, the expected cost of the lower-cost action at depth D is instead

$$\kappa_l \sum_{d=1}^{D} (1 - p_l)^{d-1} = \kappa_l \frac{1 - (1 - p_l)^D}{p_l},$$

and the comparison must include both expected worker-side cost and remaining first-passage failure probability under (8)*. Within each attempt, the lower-cost worker may take more edit steps to reach first success; that slower first-passage time is already reflected in κ_l.*

The multi-mode case is where the four-term identity matters. A lower-cost worker can be rational on one diagnostic state and irrational on another. A router is useful only when different actions win the cost–success tradeoff on different states and the pre-deployment information helps identify which state applies.

4 Decomposition-guided selection

The decomposition suggests a practical protocol. Rather than evaluate every complete design as an independent arm, the builder first estimates shared quantities: state-conditional success, cost, signal information, and router mismatch. Then a larger finite design menu can be scored analytically, with expensive deployment evaluations reserved for finalists and for paired validation.

Algorithm 1 (Theory-to-deployment selection protocol).

Input: Task family, checker, finite diagnostic state schema $\mathcal{S}$, action menu $\mathcal{A}$, admissible progressive signals $\{Z_j\}$, pilot size n_0, selected-run repeats Q, confirmation budget.

Output: Confirmed deployment policy, four accounting terms, paired deployment gain, and residual diagnostics.

1. *Freeze the state and loss.* Declare $\mathcal{S}$, success threshold δ, horizon H, cost normalizers, deployment loss (8), measurement costs, and router output contract.
2. *Structured pilot.* For each action $a \in \mathcal{A}$ and state $s \in \mathcal{S}$, run verified trajectories and estimate $\hat{p}(a, s)$, $\hat{\kappa}(a)$, first-hit curves, occupancy, and uncertainty intervals.
3. *Signal diagnostics.* For each progressive signal Z_j, estimate how much it separates the predeclared modes and log the worker allocation induced by the router.
4. *Accounting score.* Compute the four log-effort terms from Theorem 1, or the progressive-signal analogue in Appendix C.
5. *Paired deployment validation.* On the same holdout instances, run the selected action under Z_0 and richer Z_j conditions, score repeated selected-worker trajectories by (8), and subtract measurement cost.
6. *Decision.* Promote a deployment claim only if the recommended policy is stable under uncertainty checks, beats negative controls, and has acceptable residuals against the log-effort prediction.

4.1 Progressive signals and paired router value

Let Z_0 be the static signal observed before choosing a deployment action. For any admissible richer condition j, the router observes Z_j, pays the corresponding measurement and router-context cost, and selects an action

$$\hat{a}_R^j(X) = R(Z_j(X); U_R) \in \mathcal{A}.$$

Thus the experimental comparison is between routing policies ρ_0, ρ_j induced by different signals for the same orchestrator R, not between different orchestrators. The normalized measurement loss is

$$\ell_{\text{meas}}(j, x; \zeta_j) = \lambda_{\text{meas}}^\top d_j(x; \zeta_j),$$

where d_j contains measurement wall-clock, tokens, checker calls, context expansion, parsing, retry, and latency costs. Set $\ell_{\text{meas}}(0, x) = 0$.

The selected-action deployment objective is

$$\mathcal{J}_R(j) := \mathbb{E}\left[\ell_{\text{meas}}(j, X; \zeta_j) + \ell_{\text{dep}}(\hat{a}_R^j(X), X; \xi)\right]. \tag{6}$$

The router-facing value of signal condition j is the paired gain

$$\Delta_R(j) := \mathcal{J}_R(0) - \mathcal{J}_R(j) = \mathbb{E}\Big[\ell_{\text{dep}}(\hat{a}_R^0(X), X) - \ell_{\text{dep}}(\hat{a}_R^j(X), X) - \ell_{\text{meas}}(j, X)\Big]. \tag{7}$$

Positive $\Delta_R(j)$ means that the richer signal improves net deployment loss for the same router on the same task distribution. Allocation shift alone is insufficient:

$$\text{Shift}_R(j) = \Pr[\hat{a}_R^j(X) \neq \hat{a}_R^0(X)]$$

measures decision relevance, not benefit.

Proposition 1 (Unbiased paired router-gain estimator)**.** *Assume task instances are sampled i.i.d. from μ, the orchestrator, action menu, and signal protocols are fixed before evaluation, measurement costs are recorded without bias, and deployment seeds are independent conditional on selected action and instance. For condition j, estimate*

$$\widehat{\Delta}_R(j) = \frac{1}{N}\sum_{i=1}^{N}\left[\frac{1}{Q}\sum_{q=1}^{Q}\ell_{\text{dep}}(\hat{a}_{R,i}^0, x_i; \xi_{i,q}^0) - \frac{1}{Q}\sum_{q=1}^{Q}\ell_{\text{dep}}(\hat{a}_{R,i}^j, x_i; \xi_{i,q}^j) - \widehat{\ell}_{\text{meas}}(j, x_i)\right].$$

If the repeated-run averages are unbiased for the expected deployment losses of the selected actions, then $\mathbb{E}[\widehat{\Delta}_R(j)] = \Delta_R(j)$.

When the same worker is selected under Z_0 and Z_j, using common deployment seeds preserves unbiasedness and reduces paired variance. When counterfactual outcomes for all actions are logged on the same instance, hindsight action regret can be reported as a diagnostic, but it is not the primary estimand.

Table 1: Frozen operational configuration for the main campaign. All values are fixed within a task mode; repeated runs vary only the agentic trajectory.

Component	Fixed value
Dataset and split	CIFAR-10, 50,000 training examples and 10,000 validation examples; balanced subset, label noise 0.0, imbalance ratio 1.0; the train/validation split is fixed in the main campaign
Data seed	Common predeclared data/subset seed for the controlled campaign; alternative split seeds are reported only as robustness variants
`train.py` initialization seed	`SEED=42` in the main campaign; used for Python, NumPy, PyTorch, CUDA seeds, deterministic PyTorch mode, and the training-loader generator; alternative initialization seeds define robustness subinstances
Training transforms	random crop with padding 4, random horizontal flip, CIFAR-10 normalization; validation uses normalization only
Training budget per checker call	fixed-step mode with `AUTOSEARCH_MAX_STEPS=128`; time budget `120s` is a fallback, not the primary scoring convention
Common hyperparameters	`BATCH_SIZE=64`, `NUM_WORKERS=0`, Adam, learning rate $5 \cdot 10^{-4}$, weight decay 10^{-4}, betas $(0.9, 0.999)$, no LR schedule
Starting workload family	`mlp_flat`, `cnn_compact`, or `resnet_tiny`
Shared template parameters	depth 2, base channels 12, channel multiplier 2, batch norm enabled, dropout 0.0, and hidden width 48
Agent horizon	$H = 24$ edit–verify steps; trajectories continue to H after first success to measure persistence and final quality

5 Operational experimental campaign

This section specifies the controlled campaign used to test the routed-policy decomposition. The aim is not to estimate population performance over many unrelated CIFAR-10 programs. Instead, we freeze three canonical AutoResearch subinstances and use repeated agentic trajectories to estimate, for the same starting executable artifact, how worker choice, signal exposure, and routing affect first-passage success and cost.

Canonical task modes. The task is AutoResearch-style executable-artifact optimization on CIFAR-10. Each task instance contains an editable `train.py`, a read-only data/checker module `prepare.py`, and an external verifier that executes `train.py` and reports validation loss, validation accuracy, runtime, optimizer steps, and parameter count. In this campaign, a task mode is a predeclared executable optimization subinstance, not merely an architecture label. Concretely, a mode family is specified by the tuple

$$\chi_s = (A_s, D, \mathrm{split}, u, \eta_s, V, B_V, H, \delta),$$

where A_s is the starting model class and editable `train.py` template, D is CIFAR-10, split is the fixed train/validation split, u is the model-initialization seed, η_s denotes starting optimization and architecture hyperparameters, V is the checker, B_V is the checker budget, H is the agent horizon, and δ is the success threshold. The main campaign instantiates three such mode families:

$$\mathcal{S} = \{\texttt{mlp\_flat}, \texttt{cnn\_compact}, \texttt{resnet\_tiny}\}.$$

Thus there is one canonical starting artifact per mode family in the main campaign. The mode is not a worker label and is not an oracle for the best worker; it is a predeclared partition of the same AutoResearch task. It is useful only if the resulting trajectories show mode-conditional differences in first-pass competence, cost, or allocation regret.

Frozen executable configuration. Table 1 reports the fixed configuration read from the current `train.py` templates and the CIFAR-10 checker. The main campaign freezes the data split, starting artifact, training initialization seed, checker budget, success threshold, and horizon inside each mode family. This isolates the stochasticity of the agentic edit trajectory. Changing the model-initialization seed, the starting model class, or the data-subset seed creates a different executable subinstance and is therefore reserved for explicit robustness experiments.

Workers and repeated runs. The paper-facing worker menu is

$$\mathcal{M} = \{\texttt{gpt\_5\_3\_codex\_spark},\ \texttt{gpt\_5\_3\_codex},\ \texttt{gpt\_5\_4}\}.$$

Each worker uses the same prompt program, structured-edit protocol, JSON schema, tool policy, retry rule, and resource-accounting convention; the resolved model ID, CLI/API version, date, prompt hash, timeout, and token limits are logged. For every mode–worker cell we run

$$Q = 10$$

independent trajectories from the same canonical `train.py`. The repeated-run index changes only the agentic trajectory: LLM sampling, transport nondeterminism, edit proposals, and any tool-level nondeterminism. It does not change the CIFAR-10 split, the unmodified artifact, the `train.py` initialization seed, the checker budget, or the horizon. If a backend exposes an explicit sampling seed, the seed is recorded; otherwise repeated calls under the same frozen configuration are treated as independent deployment attempts and logged by run identifier.

Progressive router signals. The router is a fixed prompt-only orchestrator that chooses one worker from $\mathcal{M}$ before deployment. It never edits `train.py`. We evaluate a nested signal sequence

$$Z_0 \preceq Z_1 \preceq Z_2 \preceq Z_3.$$

Z_0 contains only the task name, checker objective, worker menu, and budget. Z_1 adds the initial `train.py` and a static workload summary. Z_2 adds a low-cost checker probe of the unmodified artifact at a predeclared probe budget. Z_3 adds the full unmodified-artifact baseline under the deployment checker budget. Probe records contain validation loss, validation accuracy, training seconds, total seconds, optimizer steps, parameter count, parse/runtime status, and the probe budget. The selected worker always starts from the original artifact; probe information is router context, not a warm start for the worker. Probe and router-context costs are charged when computing net deployment gain.

Success, cost, and deployment loss. Let $L_{\mathrm{base}}(s)$ be the validation loss of the unmodified canonical artifact for mode s under the fixed deployment checker. At edit step t, a worker produces a candidate `train.py`, the checker returns loss L_t, and checked progress is

$$G_t = \mathrm{clip}_{[0,1]}\left(\frac{L_{\mathrm{base}}(s) - L_t}{|L_{\mathrm{base}}(s)| + \varepsilon_L}\right).$$

The primary success event is first passage above the relative-improvement threshold

$$\tau_\delta = \inf\{t \le H : G_t \ge \delta\}, \qquad \delta = 0.05.$$

The competence estimator is $\hat{p}(a,s) = Q^{-1}\sum_{r=1}^{Q} \mathbf{1}\{\tau_{\delta,a,s,r} \le H\}$. The primary cost $\hat{\kappa}(a,s)$ is worker wall-clock accumulated to $\tau_\delta \wedge H$; token count is reported as a sensitivity analysis. Deployment loss is the predeclared combination in (8), using first-passage failure, threshold occupancy, final checked quality, and worker-side cost.

Theory-to-experiment map. The mathematical results are not proved by the experiments; their proofs hold under the stated assumptions. The campaign tests whether those assumptions are operationally meaningful in the frozen AutoResearch setting, whether the terms in the theory are measurable, and whether their measured changes predict deployment outcomes. Experiments 1–4 operationalize Theorem 1: Experiment 1 estimates the worker-side cost and competence terms, Experiment 2 audits the routing-information channel induced by progressive signals, Experiment 3 diagnoses routing mismatch through allocation regret, and Experiment 4 validates the resulting routed policy by paired deployment loss. Theorem 2 is tested as a single-mode subanalysis of Experiment 1, using the estimated first-passage probabilities and costs to ask when repeated deployments of a lower-cost worker match or beat one deployment of a stronger worker. Proposition 1 supplies the estimator used in Experiment 4 for the net signal-conditioned routing gain $\Delta_R(j)$. Experiments 5–6 are falsification and robustness checks: the negative controls break the relationship between Z and the task mode, while the seed perturbations probe whether Assumption 1 remains credible beyond the canonical executable artifacts.

Experiment 1: worker frontier. Run all workers on all three canonical modes without a router:

$$3 \text{ modes} \times 3 \text{ workers} \times 10 \text{ runs} = 90$$

full trajectories. Estimate $\hat{p}(a, s)$, $\hat{\kappa}(a, s)$, mean first-hit step, occupancy, final quality, and uncertainty intervals. For each mode, construct the empirical cost–success frontier

$$a^\star(s) \in \arg\min_{a \in \mathcal{M}} \{\log \hat{\kappa}(a, s) - \log \hat{p}(a, s)\}.$$

This experiment is the worker-side calibration for Theorem 1. It estimates the quantities that enter the cost and competence terms, namely $\log \kappa(M_Z)$ and $-\log p_{M_Z}(S)$, before any router is allowed to choose. The task-mode schema is useful only if different executable subinstances induce different mode-conditional frontiers; otherwise the competence term collapses to a global worker ranking. The same estimates also instantiate Theorem 2, because the first-passage success probabilities and first-passage costs determine when repeated deployments of a lower-cost worker can match the one-attempt success probability of a stronger worker.

Experiment 2: router-facing signal audit. For each canonical mode and each signal Z_j, query the same orchestrator under the same worker menu. The router returns a structured decision record containing selected worker, confidence, short task diagnosis, evidence used, expected failure mode, and expected cost risk. No external classifier is introduced. The diagnostic question is whether progressive signals change the router's own decision record: selected-worker distribution, confidence, diagnosis consistency with the predeclared mode, and cited evidence. This experiment targets the routing-information term $\mathrm{I}_\pi(S; Z)$ in Theorem 1. In the benchmark we do not give the orchestrator an oracle mode label or train a separate diagnostic classifier; instead, Z_j is the progressively richer pre-deployment record actually available to the router. Thus the audit asks whether additional signals expose solver-relevant structure of the task instance strongly enough to alter the router-facing state and selected-worker distribution. By itself this is not a deployment claim: it only tests whether the information term has an operational channel through which it could affect routing.

Experiment 3: routing shift and allocation regret. Using the decisions from Experiment 2, compute shift rates

$$\Pr[\rho_j(X) \neq \rho_0(X)]$$

across modes and router repetitions. Then use the frontier from Experiment 1 to score each selected action by allocation regret

$$r_j(s) = \big[\log \hat{\kappa}(\rho_j(s), s) - \log \hat{p}(\rho_j(s), s)\big] - \min_{a \in \mathcal{M}} \big[\log \hat{\kappa}(a, s) - \log \hat{p}(a, s)\big].$$

This is the finite-benchmark diagnostic for the routing-mismatch term $\mathbb{E}_Z \, \mathrm{KL}(\pi_Z \| q_Z)$ in Theorem 1 and for the progressive-signal comparison in Appendix C. Shift rate measures whether Z_j changes the routing policy, while allocation regret asks whether the changed policy moves toward the empirical mode-conditional frontier from Experiment 1. A signal can be informative in the sense of Experiment 2 and still have high mismatch if it makes the orchestrator change worker in the wrong direction. Conversely, low regret is the operational evidence that the richer signal improved the allocation summary q_Z rather than merely increasing decision variability.

Experiment 4: paired selected-worker deployment gain. Choose the best-performing non-static signal condition from the diagnostic split, or report all $j \in \{1, 2, 3\}$ with multiplicity control. For each mode and paired run index r, compare the worker selected under Z_0 with the worker selected under Z_j from the same original artifact and horizon. Estimate

$$\widehat{\Delta}_R(j) = \widehat{\ell}_{\mathrm{dep}}(\rho_0) - \widehat{\ell}_{\mathrm{dep}}(\rho_j) - \widehat{\ell}_{\mathrm{meas}}(j).$$

Report paired confidence intervals over run indices, first-pass success changes, cost-to-success changes, gross loss reduction, measurement cost, and the subset where the signal changed the worker but increased loss. This experiment is the deployment validation of the theory-facing diagnostics. Proposition 1 gives the estimand and estimator for the same-orchestrator comparison, while (8) defines the loss used to decide whether a signal-conditioned policy is actually better after measurement cost is charged. Positive routing information and lower mismatch are not sufficient by themselves; the routed-policy claim is supported only when the paired gain $\widehat{\Delta}_R(j)$ is positive on held-out trajectories.

Experiment 5: negative controls. Repeat Experiments 2–4 with schema-preserving controls: shuffled probe records, probes from the wrong canonical mode, and synthetic noise records with the same field names and token length. The same router, prompt, worker menu, and scoring rule are used. These controls test whether the measured routing-information and mismatch effects are specific to task-relevant signal content. They preserve the format and approximate context burden of Z_j while breaking its relationship to the task mode and worker frontier. A credible interpretation of Theorem 1 requires the real signal to reduce allocation regret and improve paired deployment gain more than these controls after measurement cost is charged.

Experiment 6: robustness to task-instance variation. The main campaign deliberately fixes `train.py` initialization and data split to isolate agentic trajectory variance. As a robustness appendix, repeat the worker-frontier and paired-routing comparisons with a small grid of additional initialization/data seeds, e.g. three seeds and five trajectories per mode–worker cell. These runs are not pooled with the main estimates. They stress-test Assumption 1, which requires the finite mode abstraction to have stable worker success probabilities and costs within each mode family. The additional seeds define nearby executable task instances, not new pooled evidence for the main canonical estimates. If competence frontiers, allocation regret, or paired gains change qualitatively under these modest perturbations, the experiment has found a useful warning: the packed-mode abstraction is too coarse for the claimed theory-facing explanation.

Claim promotion gates. The final paper should promote a routing recommendation only if all of the following are reported: state-conditional competence with uncertainty, worker-side cost sensitivity, router shift, allocation regret, paired deployment gain with measurement cost charged, negative controls, and residual diagnostics for Theorem 1. If routing diagnostics improve but paired deployment gain does not, the conclusion is diagnostic rather than prescriptive. If paired deployment gain improves but decomposition residuals are large, the protocol found a useful router but did not validate the packed-mode explanation.

6 Related work

The closest classical ancestor is algorithm selection. Since Rice's formulation, solver portfolios have treated performance as instance-dependent: SATzilla and AutoFolio learn to select solvers from instance features rather than report one globally best solver [Rice, 1976, Xu et al., 2008, Lindauer et al., 2015, Kotthoff, 2016]. That literature also includes feature-computation costs, pre-solving, and probing. Our contribution is the specialization to externally checked LLM agents, where a router observes a controlled pre-deployment record, selects a worker or harness action, and is scored by paired deployment loss rather than by standalone routing accuracy.

LLM routing and cascades provide the model-menu context. FrugalGPT frames model cascades as cost reduction across heterogeneous APIs, RouteLLM learns cost-quality routing from preference data, and recent compound-system routing work extends the idea to multi-module systems [Chen et al., 2023, Ong et al., 2024, Chen et al., 2025, Yue et al., 2025, Zaharia et al., 2024]. These methods establish that heterogeneous model menus can create economic gains. The present paper asks how much checked, task-specific information is worth before committing to an expensive agentic deployment.

Checked agent benchmarks provide the task substrate. SWE-bench evaluates repository repair against tests, AutoResearch evaluates training-loop edits against validation feedback, and theorem-proving systems use analogous external validators [Jimenez et al., 2024, Karpathy, 2026]. Most benchmarks ask how well a worker performs after it is launched. Our question is upstream: which worker should be launched, what should the router be allowed to observe, and how should the cost of that observation be charged?

Finally, repeated sampling and inference-time scaling show that lower-cost or smaller models can become competitive when verification is available [Brown et al., 2024, Snell et al., 2025, Wu et al., 2025, Zhang et al., 2024]. The first-passage retry crossover in Theorem 2 is the simplest version of that tradeoff. The four-term identity embeds the same idea in a mode-aware routing and measurement account.

7 Discussion targets and limitations

The draft has two levels of claim. The theory-facing claim is an accounting identity under Assumption 1. It is exact, but only for the certified log-effort surrogate. The deployment-facing claim is empirical: the accounting should predict or explain useful action selection under a richer deployment loss. These two claims should not be collapsed.

The packed-mode assumption is the main modeling limitation. Real agent trajectories are stateful, attempts are not independent, and diagnostic states may be fuzzy. The paper therefore reports first-passage curves, occupancy, hazard diagnostics, and residuals instead of assuming that the finite-mode surrogate is correct. If residuals dominate, the decomposition remains a useful diagnostic language but not a reliable selection rule.

The operational state schema is also a limitation. If S is too coarse, state-conditional competence is washed out and the router appears useless. If S is too fine, sample sizes per cell collapse and the signal–mode estimates become unstable. The final campaign should therefore include an ablation over state schemas, including task-family-only, probe-bin-only, and crossed schemas.

Finally, measurement value is router-specific. A baseline probe can contain information about the best action and still fail to improve deployment loss if the actual router does not use it well or if the measurement is too expensive. For this reason, the paired gain $\Delta_R(j)$ is the main deployment estimand, while the mutual-information and mismatch terms explain why that gain appears or fails to appear.

Expected final takeaway. If the final campaign passes the claim-promotion gates, the paper's message is that model choice inside an agent harness is not a scalar leaderboard problem. It is a measurable decomposition problem validated by paired deployment outcomes. The right worker depends on task state, checked competence, routing information, routing mismatch, first-passage time, retry depth, and worker-side cost.

References

Alessandro Achille and Stefano Soatto. AI agents as universal task solvers: It's all about time. *Entropy*, 28(3):332, 2026.

Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V. Le, Christopher Ré, and Azalia Mirhoseini. Large language monkeys: Scaling inference compute with repeated sampling. *arXiv preprint arXiv:2407.21787*, 2024.

Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176*, 2023.

Lingjiao Chen et al. Optimizing model selection for compound AI systems. *arXiv preprint arXiv:2502.14815*, 2025.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.

Andrej Karpathy. autoresearch: AI agents running research on single-GPU nanochat training automatically. `https://github.com/karpathy/autoresearch`, 2026. GitHub repository.

Lars Kotthoff. Algorithm selection for combinatorial search problems: A survey. *AI Magazine*, 35 (3):48–60, 2016.

Marius Lindauer, Katharina Eggensperger, Matthias Feurer, Bernd Bischl, and Frank Hutter. AutoFolio: An automatically configured algorithm selector. *Journal of Artificial Intelligence Research*, 53: 745–778, 2015.

Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M Waleed Kadous, and Ion Stoica. RouteLLM: Learning to route LLMs with preference data. *arXiv preprint arXiv:2406.18665*, 2024.

John R. Rice. The algorithm selection problem. In *Advances in Computers*, volume 15, pages 65–118. Elsevier, 1976.

Charlie Victor Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. In *International Conference on Learning Representations (ICLR)*, 2025.

Vladimir N. Vapnik. Structure of statistical learning theory. In Alexander Gammerman, editor, *Computational Learning and Probabilistic Reasoning*, pages 33–41. John Wiley and Sons, 1996.

Yangzhen Wu, Zhiqing Sun, Shanda Li, Sean Welleck, and Yiming Yang. Inference scaling laws: An empirical analysis of compute-optimal inference for problem-solving with language models. In *International Conference on Learning Representations (ICLR)*, 2025.

Lin Xu, Frank Hutter, Holger H. Hoos, and Kevin Leyton-Brown. SATzilla: Portfolio-based algorithm selection for SAT. *Journal of Artificial Intelligence Research*, 32:565–606, 2008.

Yanwei Yue, Guibin Zhang, Boyang Liu, Guancheng Wan, Kun Wang, Dawei Cheng, and Yiyan Qi. MasRouter: Learning to route LLMs for multi-agent systems. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15549–15572, 2025.

Matei Zaharia, Omar Khattab, Lingjiao Chen, Jared Quincy Davis, Heather Miller, Chris Potts, James Zou, Michael Carbin, Jonathan Frankle, Naveen Rao, and Ali Ghodsi. The shift from models to compound AI systems. Berkeley Artificial Intelligence Research Blog, 2024. URL `https://bair.berkeley.edu/blog/2024/02/18/compound-ai-systems/`.

Jinghan Zhang, Kunhao Zheng, Yici Yan, Yuchen Ouyang, and Jun Zhu. Scaling LLM inference with optimized sample compute allocation. *arXiv preprint arXiv:2410.22480*, 2024.

A Proof details

A.1 Four-term identity

Under Assumption 1,

$$\log T_{\rho,q}(S,Z) = C + \log \kappa(M_Z) - \log p_{M_Z}(S) - \log q_Z(S).$$

Conditioning on $Z = z$,

$$\mathbb{E}[-\log q_z(S) \mid Z = z] = \sum_s \pi_z(s) \log \frac{1}{q_z(s)} = \mathrm{H}(\pi_z) + \mathrm{KL}(\pi_z \| q_z).$$

Therefore

$$\mathbb{E}[\log T_{\rho,q}] = C + \mathbb{E}_Z[\log \kappa(M_Z)] + \mathbb{E}_Z \mathbb{E}_{S\sim\pi_Z}[-\log p_{M_Z}(S)] + \mathbb{E}_Z[\mathrm{H}(\pi_Z)] + \mathbb{E}_Z[\mathrm{KL}(\pi_Z \| q_Z)].$$

For the prior-matched baseline ρ_0 that deploys M_0 and uses $q_z = \pi$,

$$\mathbb{E}[\log T_{\rho_0,\pi}] = C + \log \kappa(M_0) + \mathbb{E}_\pi[-\log p_{M_0}(S)] + \mathrm{H}(\pi).$$

Subtracting the deployed effort from the baseline effort yields

$$\Delta = \mathbb{E}_Z\left[\log \frac{\kappa(M_0)}{\kappa(M_Z)}\right] + \mathbb{E}_Z \mathbb{E}_{S\sim\pi_Z}\left[\log \frac{p_{M_Z}(S)}{p_{M_0}(S)}\right] + \mathrm{H}(\pi) - \mathbb{E}_Z[\mathrm{H}(\pi_Z)] - \mathbb{E}_Z[\mathrm{KL}(\pi_Z \| q_Z)].$$

The entropy difference is $\mathrm{I}_\pi(S;Z)$, giving (5).

A.2 Pilot concentration

For binary success observations in an action-state cell, Hoeffding gives

$$\mathbb{P}(|\hat{p}(a,s) - p(a,s)| > \alpha) \le 2\exp(-2n_0\alpha^2).$$

If all true probabilities are at least $p_{\min}$, then the log-probability error obeys $|\log \hat{p} - \log p| \le \alpha/p_{\min} + O(\alpha^2/p_{\min}^2)$ on the good event. A union bound over $|\mathcal{A}|\,|\mathcal{S}|$ cells yields the conservative pilot rule

$$n_0 = O\left(\frac{\log(|\mathcal{A}|\,|\mathcal{S}|/\delta_{\mathrm{conf}})}{\delta_{\mathrm{acc}}^2 p_{\min}^2}\right).$$

Wilson intervals and bootstrap intervals are reported in practice because pilot cell counts are small.

B Operational diagnostics for the four terms

The identity above becomes useful only after each term is tied to quantities logged by the deployment protocol. The checker-facing progress process is the common object used to operationalize competence, cost, information, and mismatch. Fix a horizon H and a task-normalized checked progress process $G_t(a, x) \in [0, 1]$ for action a on instance x. In the main experiment, the router acts once before deployment: after observing Z, it selects a worker M, and the same worker is used for the whole trajectory. The success and cost diagnostics below are therefore trajectory-level quantities computed from the step-wise verifier trace. For lower-is-better checker losses, one admissible normalization is

$$G_t(a, x) = \mathrm{clip}_{[0,1]}\left(\frac{L_0(x) - L_t(a, x)}{|L_0(x)| + \varepsilon_L}\right),$$

where $L_0(x)$ is the starting-artifact loss and $L_t(a, x)$ is the checked loss convention at step t. Given a success threshold $\delta > 0$,

$$\tau_\delta(a, x) = \inf\{t \le H : G_t(a, x) \ge \delta\}, \qquad F_\delta(a, x) = \mathbf{1}\{\tau_\delta(a, x) = \infty\}.$$

We also log persistence through

$$\mathrm{Occ}_{\delta,H}(a, x) = \frac{1}{H}\sum_{t=1}^{H} \mathbf{1}\{G_t(a, x) \ge \delta\}.$$

Competence. The worker-level competence component $p_M(s)$ is the first-success probability of worker or action M on task mode s:

$$p_M(s) = \mathbb{P}[\tau_\delta(M, X) \le H \mid S = s].$$

Thus G_t is step-wise, but the policy competence term uses the probability that the worker selected by the router reaches the success threshold at least once before the horizon. Cumulative progress and persistence are not substituted for $p_M(s)$ in the identity. We log

$$\bar{G}_M(s) = \mathbb{E}\left[\frac{1}{H}\sum_{t=1}^{H} G_t(M, X) \mid S = s\right]$$

and $\mathrm{Occ}_{\delta,H}$ as secondary diagnostics, especially when first-hit success saturates or when partial progress is scientifically relevant.

Cost. The worker-level cost component $\kappa(M)$ is kept separate from competence. In the main protocol it is the normalized cost of selecting worker M for the same routed trajectory used to define $p_M(s)$; the routed-policy cost term averages this quantity over the workers selected under Z. The primary convention is cost accumulated until the first certified success or the horizon, $\tau_\delta \wedge H$. We log

$$c_\delta(a, x; \xi) = \big(\widetilde{T}^{\mathrm{worker}}_{\tau_\delta \wedge H}, \widetilde{C}^{\mathrm{tok}}_{\tau_\delta \wedge H}\big)(a, x; \xi),$$

and report worker wall-clock cost as primary, with token count as a sensitivity analysis. Checker runtime is logged for audit, but it is not part of $\kappa(M)$ unless the checker protocol or checker-call frequency differs across workers.

Routing information. The theorem term $\mathrm{I}(S; Z)$ measures how much a router-visible signal Z separates the predeclared task modes before deployment. This does not mean that the evaluator already knows the best worker for each instance. The evaluator knows only the candidate mode schema and can estimate whether Z_j makes that schema more identifiable:

$$\mathrm{I}(S; Z_j \mid Z_0) = \mathrm{H}(S \mid Z_0) - \mathrm{H}(S \mid Z_j).$$

The quantity is estimated from held-out instances using predeclared diagnostic mode labels and logged signal records. It is a descriptive information diagnostic; its decision value is established only after worker outcomes show that the modes predict different cost–success profiles. The paired router experiment then checks whether higher-information signals actually change the selected worker and improve first-passage deployment outcomes.

Routing mismatch and allocation regret. The theorem term $\mathbb{E}_Z \mathrm{KL}(\pi_Z \| q_Z)$ measures whether the router uses the mode-relevant information made available by Z. Operationally, the router returns a worker, not a mode label or distribution. Thus q_Z is treated as an evaluator-side summary of the worker allocation induced by the router under signal Z. After deployment runs, the evaluator estimates the mode-conditional frontier

$$a^\star(s) \in \arg\min_{a \in \mathcal{A}} \{\log \kappa(a) - \log p(a, s)\},$$

or the statistically indistinguishable set of frontier workers. For a selected action $a = \rho_j(x)$ and mode $s = \sigma(x)$, the corresponding allocation regret diagnostic is

$$r_j(x) = \big[\log \kappa(a) - \log p(a, s)\big] - \min_{a' \in \mathcal{A}} \big[\log \kappa(a') - \log p(a', s)\big].$$

This regret is computed retrospectively on a diagnostic split and validated on holdout instances; it is not an oracle input to the router. High information with high mismatch means that the progressive signal exposed useful mode structure, but the induced worker allocation did not exploit the empirical cost–success frontier.

After these four terms are estimated, the experiment still reports a decision-facing deployment loss as final validation. This loss is not a replacement for the theorem; it tests whether designs favored or explained by the four diagnostics improve the practical deployment objective. We predeclare

$$\begin{aligned} \ell_{\mathrm{dep}}(a, x; \xi) = \alpha_{\mathrm{occ}} \big(1 - \mathrm{Occ}^{\mathrm{cur}}_{\delta, H}(a, x; \xi)\big) + \alpha_{\mathrm{fail}} F_\delta(a, x; \xi) \\ + \alpha_{\mathrm{qual}} \psi(G^{\mathrm{cur}}_H(a, x; \xi)) + \lambda^\top c_\delta(a, x; \xi), \end{aligned} \tag{8}$$

which combines persistence, failure, final checked quality, and resource cost.

C Operational decomposition under progressive signals

The main four-term identity compares a deployed design to a prior-matched baseline. For the V3 router experiment, it is often useful to compare two progressive signal conditions for the same router. Assume a finite diagnostic state $S \in \mathcal{S}$ and, for each Z_j, the empirical mode distribution induced by the signal

$$\pi_j(s) = \mathbb{P}[S = s \mid Z_j].$$

The fixed orchestrator R induces the routing policy $\rho_j : Z_j \mapsto \hat{a}^j_R(Z_j)$, and the evaluator constructs a mode-facing allocation summary $q^j_R(\cdot \mid Z_j) \in \Delta(\mathcal{S})$. Define

$$\mathcal{E}^j_R(s, Z_j) = \frac{\kappa(\hat{a}^j_R(Z_j))}{p(\hat{a}^j_R(Z_j), s)\, q^j_R(s \mid Z_j)}.$$

The log-effort gain of signal condition Z_j over Z_0 is

$$\Delta^{\log}_R(j) := \mathbb{E}[\log \mathcal{E}^0_R(S, Z_0) - \log \mathcal{E}^j_R(S, Z_j)].$$

Expanding the cross-entropy terms gives

$$\Delta^{\log}_R(j) = C_R(j) + \Phi_R(j) + G(j) + M_R(j),$$

where

$$\begin{aligned} C_R(j) &= \mathbb{E}\left[\log \frac{\kappa(\hat{a}^0_R(Z_0))}{\kappa(\hat{a}^j_R(Z_j))}\right], \\ \Phi_R(j) &= \mathbb{E}\left[\log \frac{p(\hat{a}^j_R(Z_j), S)}{p(\hat{a}^0_R(Z_0), S)}\right], \\ G(j) &= \mathbb{E}[\mathrm{H}(\pi_0) - \mathrm{H}(\pi_j)], \\ M_R(j) &= \mathbb{E}\Big[\mathrm{KL}(\pi_0 \| q^0_R(\cdot \mid Z_0)) - \mathrm{KL}(\pi_j \| q^j_R(\cdot \mid Z_j))\Big]. \end{aligned}$$

When Z_j refines Z_0, $G(j) = \mathrm{I}(S; Z_j \mid Z_0)$. This signal-state decomposition is a diagnostic companion to the deployment-gain estimand (7), not a replacement for it.

D Operational estimator details

Trajectory record. Each run stores the worker alias, resolved model identifier, split, task family, diagnostic state, seed, maximum horizon H, signal condition, prompt hash, router output, selected worker, and cost convention. Each step stores the prompt-visible state, proposed `train.py` artifact, parse status, checker result, validation loss, validation accuracy, relative improvement, incremental worker wall time, token usage, and checker runtime for audit.

Success and time-to-verification. For trajectory i, τ_i is the first step at which the relative improvement threshold is crossed; censored failures have $\tau_i = \infty$. For each action-state cell,

$$\hat{F}_a(s,h) = \frac{1}{n_{a,s}} \sum_i \mathbf{1}\{\tau_i \le h\}, \qquad \hat{S}_a(s,h) = 1 - \hat{F}_a(s,h).$$

The empirical hazard is

$$\hat{\lambda}_a(s,h) = \frac{\sum_i \mathbf{1}\{\tau_i = h\}}{\sum_i \mathbf{1}\{\tau_i \ge h\}}.$$

The theorem-facing estimator used in the main protocol is the first-success probability $\hat{p}(a,s) = \hat{F}_a(s,H)$.

Cost. The primary cost is cumulative worker wall-clock time to $\tau \wedge H$. Secondary worker-side cost is cumulative token count. Checker runtime is logged as a protocol audit field and excluded from worker-cost comparisons unless the checker protocol differs across workers. All plots that support model recommendations must be reproducible under at least worker wall-clock and token-count conventions. When a worker fails before producing a parseable artifact, its cost is still counted and the step is marked unsuccessful.

Information and mismatch. For each progressive signal Z_j, estimate $\pi_j(s) = \mathbb{P}[S = s \mid Z_j]$ from the predeclared mode labels and logged signal records on held-out instances. Mismatch is

$$\widehat{\mathrm{MM}}(j) = \frac{1}{n} \sum_i \mathrm{KL}\Big(\hat{\pi}_j(\cdot \mid Z_{j,i}) \Big\| \hat{q}^j_R(\cdot \mid Z_{j,i})\Big).$$

The report should also map both mode distributions through state-action scores to obtain induced action regret, since a small mismatch can still matter when the confused states have different best workers.

Uncertainty. Success probabilities use Wilson intervals. Aggregated objectives, information, mismatch, paired gains, and residuals use bootstrap or randomization tests at the task-instance level. Because the number of task-family clusters may be small, paired randomization tests are primary for $\Delta_R(j)$ and bootstrap intervals are descriptive unless the expanded campaign has enough independent clusters.

E Checklist notes for finalization

Before submission, the checklist should be updated to reflect the final state of the experiments. The current draft contains the theory, operational protocol, statistical plan, and falsification criteria needed for a full NeurIPS-style writeup, but claims should cite exact frozen results only after the final campaign is complete.

NeurIPS Paper Checklist

The checklist is designed to encourage best practices for responsible machine learning research, addressing issues of reproducibility, transparency, research ethics, and societal impact. Do not remove the checklist: **The papers not including the checklist will be desk rejected.** The checklist should follow the references and follow the (optional) supplemental material. The checklist does NOT count towards the page limit.

Please read the checklist guidelines carefully for information on how to answer these questions. For each question in the checklist:

- You should answer [Yes], [No], or [N/A].
- [N/A] means either that the question is Not Applicable for that particular paper or the relevant information is Not Available.
- Please provide a short (1–2 sentence) justification right after your answer (even for [N/A]).

The checklist answers are an integral part of your paper submission. They are visible to the reviewers, area chairs, senior area chairs, and ethics reviewers. You will also be asked to include it (after eventual revisions) with the final version of your paper, and its final version will be published with the paper.

The reviewers of your paper will be asked to use the checklist as one of the factors in their evaluation. While [Yes] is generally preferable to [No], it is perfectly acceptable to answer [No] provided a proper justification is given (e.g., error bars are not reported because it would be too computationally expensive" or "we were unable to find the license for the dataset we used"). In general, answering [No] or [N/A] is not grounds for rejection. While the questions are phrased in a binary way, we acknowledge that the true answer is often more nuanced, so please just use your best judgment and write a justification to elaborate. All supporting evidence can appear either in the main paper or the supplemental material, provided in appendix. If you answer [Yes] to a question, in the justification please point to the section(s) where related material for the question can be found.

IMPORTANT, please:

- **Delete this instruction block, but keep the section heading "NeurIPS Paper Checklist"**,
- **Keep the checklist subsection headings, questions/answers and guidelines below.**
- **Do not modify the questions and only use the provided macros for your answers**.

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and introduction state the paper's scope: verifiable agentic systems, a packed latent-mode assumption, decomposition-guided selection, and a frozen long-horizon experimental campaign. The discussion section explicitly marks which claims require final campaign confirmation.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: Section 7 discusses the packed-mode assumption, small live-agent sample sizes, finite design menus, oracle mode labels, and current cost-measurement limitations.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate "Limitations" section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [Yes]

Justification: Assumption 1 states the assumptions for the main theoretical result, Theorem 1 includes a proof sketch, and Appendix A gives the algebraic proof and the near-optimality proof.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Section 5 describes the AutoResearch task instantiation, task modes, model menu, router-visible signals, horizons, metrics, and campaign template; Appendix D specifies the required logs and estimators. The final submission should freeze exact run IDs and commit hashes after the full campaign.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [No]

Justification: The draft is written against an existing repository and includes reproducibility commands, but an anonymized public code/data release has not yet been prepared for submission. The final submission should either include an anonymized supplement or update this answer.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.

- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes]

   Justification: Section 5 describes the AutoResearch task instantiation, task modes, model menu, router-visible signals, primary estimators, horizons, and final campaign design. Additional implementation details are listed in Appendix D.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [Yes]

   Justification: Section 5 and Appendix D specify Wilson intervals, bootstrap intervals, survival curves, residual diagnostics, and held-out evaluation. Final numerical intervals will be filled after the frozen campaign.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
   - The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
   - The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
   - The assumptions made should be given (e.g., Normally distributed errors).
   - It should be clear whether the error bar is the standard deviation or the standard error of the mean.
   - It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
   - For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
   - If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [No]

Justification: The current draft reports wall-clock costs and run counts but does not yet provide a complete compute-resource table for every final experiment. This should be added after the final campaign is executed.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

Answer: [Yes]

Justification: The work studies verifiable benchmark tasks and does not involve human subjects, sensitive personal data, or released high-risk models. The submission should preserve anonymity and follow NeurIPS code and data policies.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: Section 7 notes positive uses in cost-aware and transparent agent deployment as well as limitations and failure modes. The work is methodological and does not introduce a new generative model or dataset of people.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.

- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: [N/A]

    Justification: The paper does not release a pre-trained model, image generator, scraped dataset, or other asset with high misuse risk. It evaluates agent harnesses on synthetic or generated benchmark tasks.

    Guidelines:

    - The answer [N/A] means that the paper poses no such risks.
    - Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
    - Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
    - We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [Yes]

    Justification: The related-work section cites existing papers and systems used for positioning, and the experimental section describes the benchmark surfaces. The final submission should add explicit license details for any released code and benchmark assets.

    Guidelines:

    - The answer [N/A] means that the paper does not use existing assets.
    - The authors should cite the original paper that produced the code package or dataset.
    - The authors should state which version of the asset is used and, if possible, include a URL.
    - The name of the license (e.g., CC-BY 4.0) should be included for each asset.
    - For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
    - If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
    - For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
    - If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: The work introduces a code harness and analysis artifacts as research assets, and Appendix D sketches their documentation. A submission-ready anonymized asset card and frozen command list remain to be prepared.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The paper does not involve crowdsourcing or research with human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: The paper does not involve crowdsourcing or research with human subjects, so IRB or equivalent approval is not applicable.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [Yes]

Justification: LLMs are central to the evaluated agent designs. Section 5 describes the model menu and the six-branch LLM editing protocol; the final submission should add exact model versions and API dates for all live runs.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.